

Chat Agent — Luồng và xử lý chi tiết

YoMedia Dashboard · NovaAI Assistant

Tài liệu nội bộ · Cập nhật: 2026

1. Tổng quan

Hệ thống chat AI hoạt động theo mô hình **request/response qua tRPC** (không streaming). Hai lớp chính:

Lớp	File / thành phần	Vai trò
Client	apps/web/components/Chat.tsx	UI, lịch sử trong localStorage, chọn Gemini/OpenAI, đính kèm file
API	apps/web/lib/trpc/api.ts → rag.query	Gọi mutation có auth
Server	answerWithRag	Guardrails, routing intent/tool, RAG, LLM, short memory

2. Luồng tổng thể (sequence)





Chat.tsx hiển thị bubble assistant (+ BuildDemoProgress nếu UI nhận diện build demo)

sessionId trên server = activeConversation.id trên client.

3. Pipeline server (answerWithRag)

File: apps/server/src/lib/ai/orchestration/answerWithRag.ts

3.1 Guardrails

- Tin trống → từ chối
- Độ dài > **1500** ký tự (MAX_QUESTION_LENGTH)
- Chứa từ nhạy cảm: password , token , api key , credit card , ...

File: apps/server/src/lib/ai/guardrails/index.ts

3.2 Short memory (server)

- Key: {email hoặc role:guest}::{sessionId hoặc default}
- TTL mặc định: **2 giờ** (SHORT_MEMORY_TTL_MS)
- Tối đa **8 cặp** user/assistant (MAX_HISTORY_TURNS)
- File Build Demo pending: lưu riêng đến khi upload thành công

File: apps/server/src/lib/ai/memory/shortMemory.ts

3.3 Phát hiện tool (ưu tiên cao nhất)

resolveActionTool (detectTool.ts):

Tool	Trigger ví dụ
help	help, hướng dẫn, trợ giúp
time_now	mấy giờ, time now, giờ hiện tại
build_demo	build demo, upload demo, sftp, tải lên, ...

Phiên Build Demo đang mở: nếu history đã bắt đầu build demo, assistant đang hỏi brand/format, hoặc hasBuildDemoAttachments → mọi turn tiếp theo vẫn route build_demo .

3.4 Phân loại intent (khi không có tool)

1. **LLM classifier** → JSON { intent: knowledge_qa | free_chat, confidence, reason }
2. Nếu LLM lỗi → **rule-based** classifyUserIntent (điểm từ khóa: câu hỏi, SOP, tài liệu, ...)
3. Khi có tool → intent: actions , classifierSource: rule_tool

3.5 Ba nhánh xử lý

Intent	Hành vi
actions	executeTool hoặc runBuildDemoTool
knowledge_qa	Score file trong apps/server/rag/docs → ghép context → LLM

free_chat

LLM + system prompt, không RAG

3.6 Gọi model & fallback

- Provider: `gemini` (mặc định) hoặc `openai`
- Model: `GEMINI_CHAT_MODEL` / `OPENAI_CHAT_MODEL` hoặc mặc định `gemini-1.5-flash` / `gpt-4o-mini`
- Provider chính lỗi → thử provider còn lại (`fallbackUsed: true`)

3.7 Logging

`logChatEvent` : `chat_guardrail_block` , `chat_tool_called` , `chat_query` , `chat_provider_failed` , `chat_clear_history`

4. Nhánh Build Demo

```
runBuildDemoTool
|
├─ mergeBuildDemoAttachments(memoryKey, attachments)
|
├─ invokeBuildDemoAgent (OpenAI/Gemini function calling: build_demo)
|   ├─ kind: message → trả text hỏi thêm (thiếu brand/format/file)
|   └─ kind: tool_call → { brandId, productCateId, demoFormat, folderName? }
|
└─ executeBuildDemo
    ├─ Validate brand theo quyền user (admin = all brands)
    ├─ Decode base64 attachments
    ├─ Upload SFTP: /script/demo/{year}/{month}/...
    └─ Video: sinh VAST XML (makeVastXml.ts)
```

Quyền brand: admin → `null` (mọi brand); user → `resolveAllowedBuildDemoBrands(account)` .

Thành công: prefix `Build Demo thành công` → `clearBuildDemoAttachments` .

Files:

- `buildDemoTool.ts` — orchestration tool
- `buildDemoAgent.ts` — extraction metadata
- `buildDemoExecutor.ts` — SFTP + VAST
- `buildDemoConfig.ts` — brand / product category

Client: `BuildDemoProgress` chỉ là progress UI giả lập; server trả một response khi xong.

5. RAG (Knowledge Base)

File: `apps/server/src/lib/ai/retrieval/knowledgeBase.ts`

1. Load `.md` , `.txt` , `.json` từ `apps/server/rag/docs` (cache in-memory)
2. **scoreDoc**: khớp keyword câu hỏi với nội dung doc
3. Lấy top **3** doc, extract snippet theo dòng liên quan
4. Ghép `contextPrompt` → đưa vào LLM

Nếu không có doc / không match → `fallbackMessage` trong prompt.

6. Client (Chat.tsx)

Gửi tin

```
api.rag.query(text || "upload demo", provider, attachments, activeConversation.id)
```

Bộ nhớ

Loại	Nơi lưu	Mục đích
Conversations	localStorage key yomedia.chat.conversations.v1	UI, max 30 hội thoại
Short memory	Server RAM Map	Context LLM (8 turn)
Pending files	Server session	File nhiều lượt trước khi đủ metadata

Xóa phiên

- Client: reset messages + `api.rag.clearSession({ sessionId })`
- Xóa hội thoại cũ: `clearSession` cho từng `sessionId` đã xóa

7. API tRPC

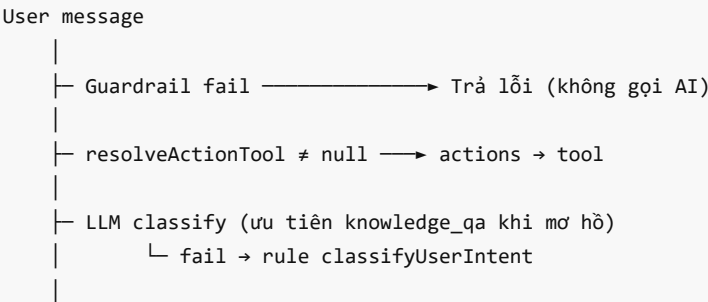
Router: `apps/server/src/trpc/routers/rag.ts`

Procedure	Mô tả
<code>rag.query</code>	mutation — xử lý chat
<code>rag.clearSession</code>	xóa short memory (một session hoặc allSessions)

Input `rag.query` : `question` , `sessionId?` , `provider?` , `attachments?`

Output (RagAnswerResult): `ok` , `answer` , `provider` , `intent` , `sources[]` , `toolCalled?` , `buildDemoProcessing?` , `fallbackUsed`

8. Sơ đồ quyết định intent



```
└─ knowledge_qa → RAG docs → LLM
└─ free_chat  ───────────→ LLM trực tiếp
```

9. Bản đồ file

Thành phần	Đường dẫn
UI Chat	apps/web/components/Chat.tsx
API client	apps/web/lib/trpc/api.ts
Router	apps/server/src/trpc/routers/rag.ts
Orchestrator	apps/server/src/lib/ai/orchestration/answerWithRag.ts
Tool routing	apps/server/src/lib/ai/actions/detectTool.ts
Actions index	apps/server/src/lib/ai/actions/index.ts
Build Demo tool	apps/server/src/lib/ai/actions/buildDemoTool.ts
Build Demo agent	apps/server/src/lib/ai/actions/buildDemoAgent.ts
Build Demo executor	apps/server/src/lib/ai/actions/buildDemoExecutor.ts
Intent (rule)	apps/server/src/lib/ai/intent/classifyUserIntent.ts
RAG retrieval	apps/server/src/lib/ai/retrieval/knowledgeBase.ts
Config / prompt	apps/server/src/lib/ai/core/config.ts
Types	apps/server/src/lib/ai/core/types.ts

10. Biến môi trường liên quan

Biến	Mục đích
OPENAI_API_KEY	Chat + Build Demo agent (OpenAI)
GEMINI_API_KEY	Chat + Build Demo agent (Gemini)
OPENAI_CHAT_MODEL	Model OpenAI chat
GEMINI_CHAT_MODEL	Model Gemini chat
CHAT_SYSTEM_PROMPT	System prompt tùy chỉnh
SHORT_MEMORY_TTL_MS	TTL short memory (ms)

Tài liệu được sinh từ codebase YoMedia Dashboard.